Searching Algorithms

Searching Algorithms

1. Linear search

```
Linear_search(arr,val)
//input: arr as Array to search within and val as value to search
Step 1: i <- 0, n <- arr.length
Step 2: if i > n-1 go to Step 6
Step 3: if arr[i] not = val then go to Step 4 else go to Step 5
Step 4: i <- i+1, go to Step 2
Step 5: Print “value found” go to Step 7
Step 6: Print “value not found”
Step 7: Exit
```

Sorting Algorithms

Sorting Algorithms:

Bubble Sort:

```
BubbleSort(arr){
      loop: i=0 to n-1
            loop: j=0 to n-i-2
                  if arr[j] > arr[j+1] then
                        swap arr[j] and arr[j+1]
            end loop
      end loop
}
```

Insertion Sort

```
InsertionSort(arr){
      loop: i=1 to n-1
            key = arr[i]
            j=i-1
            loop till key<arr[j] and j>=0 is true:
                  arr[j+1] = arr[j]
                  j=j-1
            arr[j+1] = key
}
```

Merge Sort

```
mergeSort(arr){
      if len(arr) <= 1:
            return arr
      mid = len(arr)//2
      lefthalf = arr[0:mid-1]
      righthalf = arr[mid:(len(arr)-1)]

      sortedleft = mergeSort(lefthalf)
      sortedright = mergeSort(righthalf)

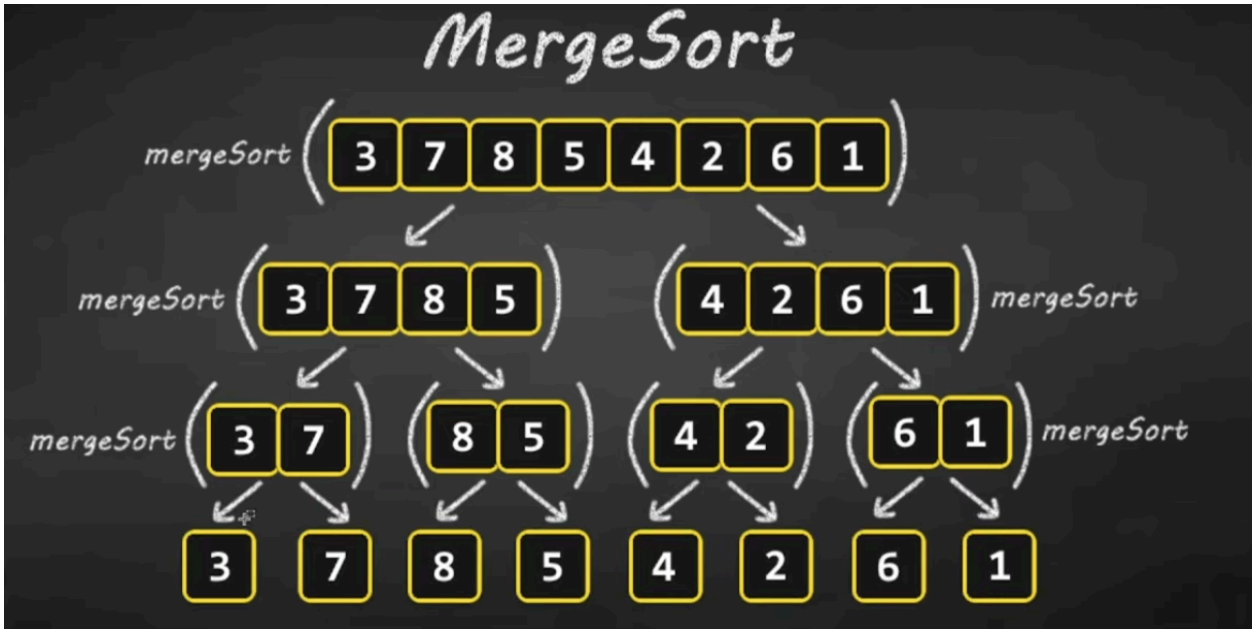
      return merge(sortedleft, sortedright)
}

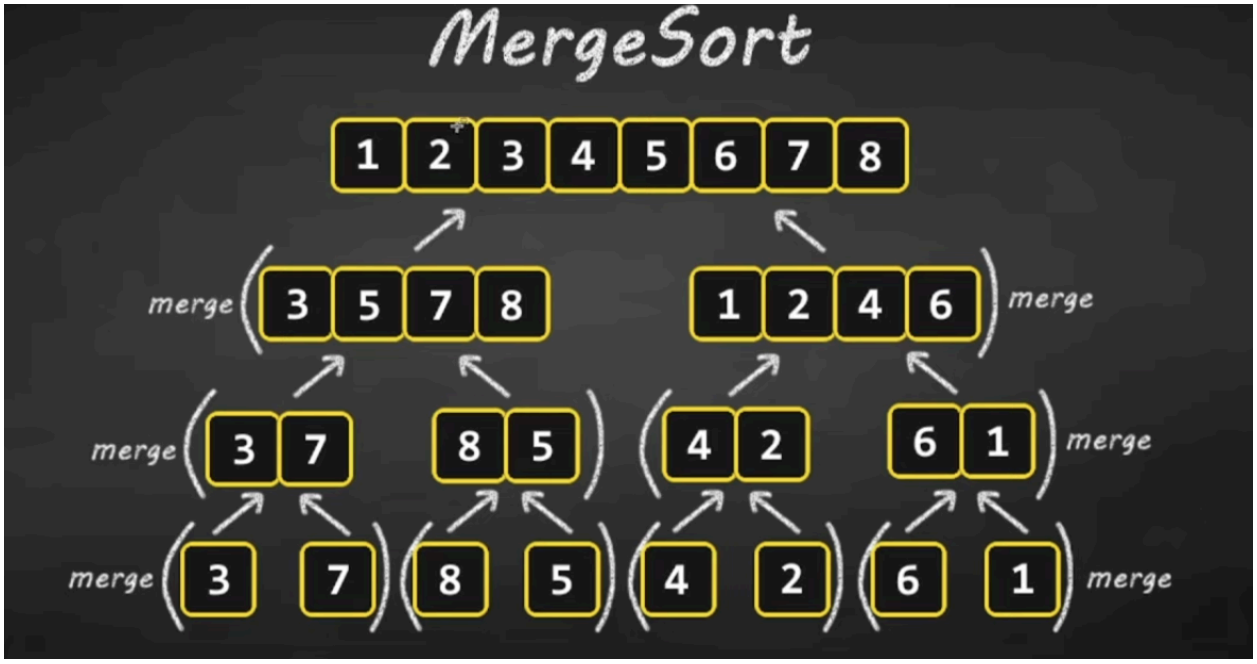
merge(left, right){
      result = []
      i=j=0

      while i < len(left) and j < len(right){

            if left[i] < right[j]:
                  add left[i] into result
                  i = i+1
            else:
                  add right[j] into result
                  j = j+1
      }
      loop: i<len(left)
            add left[i] into result
            i = i+1
      loop: j<len(right)
            add right[j] into result
            j = j+1

      return result
}
```


MergeSort
mergeSort
3 7 8 5 4 2 6 1
mergeSort
3 7 8 5
4 2 6 1
mergeSort
mergeSort
3 7
8 5
4 2
6 1
mergeSort
3 7 8 5 4 2 6 1


MergeSort
1 2 3 4 5 6 7 8
merge
3 5 7 8
1 2 4 6
merge
merge
3 7
8 5
4 2
6 1
merge
merge
3 7 8 5 4 2 6 1
merge

Quick Sort

```
Partition(arr, low, high){
      pivot = arr[high]

      loop: j=low to high-1{
            if arr[j] < pivot{
                  i+=1
                  swap(arr[i] and arr[j])
            }
      }

      swap(arr[i+1] and arr[high])
      return i+1
}


QuickSort(arr, low, high){
      if low<high{
            piv = partition(arr, low, high)

            quickSort(arr, low, piv-1)
            quickSort(arr, piv+1, high)
      }
}

QuickSort(arr, 0, len-1)
```